Slip 1

In [1]:
```
# 1 Implementing Linear Regression to predict housing prices based on features
# a. Import the “House Price Prediction” data and analyse it with the help of
# b. Carry out the required steps of data pre-processing and do necessary Visu
# c. Split the data and train the data with the Linear Regression model.
# d. Make the predictions and evaluate it on the test data.
```

In [2]:
```
# ==============================
# Step 1: Import Libraries
# ==============================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

In [3]:
```
# ==============================
# Step 2: Load Dataset
# ==============================
data = pd.read_excel("/content/mlr.xlsx")   # change path if needed

print("First 5 rows:")
print(data.head())

print("\nDataset Info:")
print(data.info())

print("\nStatistical Summary:")
print(data.describe())
```

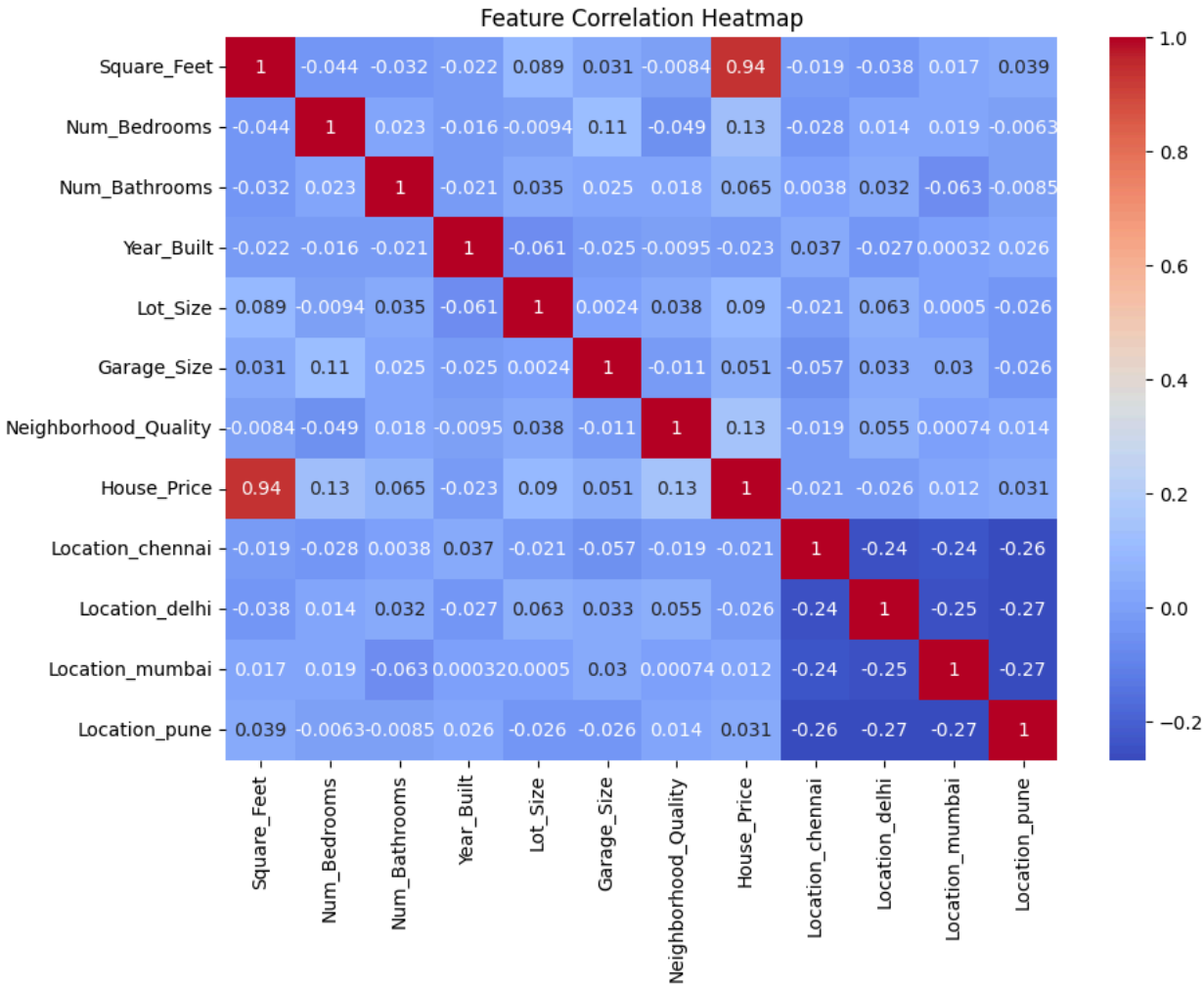
```
First 5 rows:
   Square_Feet  Num_Bedrooms  Num_Bathrooms  Year_Built  Lot_Size  \
0         1360             2              1        1981  1.088566
1         4272             3              3        2016  4.780457
2         3592             1              2        2016  3.786509
3          966             1              2        1977  2.982815
4         4926             2              1        1993  4.732509

   Garage_Size  Neighborhood_Quality   Location   House_Price
0            0                     5    chennai  6.084810e+05
1            1                     6  bangalore  1.768604e+06
2            0                     9    chennai  1.298508e+06
3            1                     8    chennai  6.517251e+05
4            0                     8     mumbai  1.918317e+06

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Square_Feet           1000 non-null   int64
 1   Num_Bedrooms          1000 non-null   int64
 2   Num_Bathrooms         1000 non-null   int64
 3   Year_Built            1000 non-null   int64
 4   Lot_Size              1000 non-null   float64
 5   Garage_Size           1000 non-null   int64
 6   Neighborhood_Quality  1000 non-null   int64
 7   Location              1000 non-null   object
 8   House_Price           1000 non-null   float64
dtypes: float64(2), int64(6), object(1)
memory usage: 70.4+ KB
None

Statistical Summary:
       Square_Feet  Num_Bedrooms  Num_Bathrooms   Year_Built     Lot_Size  \
count  1000.000000   1000.000000    1000.000000  1000.000000  1000.000000
mean   2815.422000      2.990000       1.973000  1986.550000     3.024967
std    1255.514921      1.427564       0.820332    20.632916     1.153692
min     503.000000      1.000000       1.000000  1950.000000     1.005385
25%    1749.500000      2.000000       1.000000  1969.000000     2.036396
50%    2862.500000      3.000000       2.000000  1986.000000     3.053102
75%    3849.500000      4.000000       3.000000  2004.250000     4.042949
max    4999.000000      5.000000       3.000000  2022.000000     4.990491

       Garage_Size  Neighborhood_Quality   House_Price
count  1000.000000           1000.000000  1.000000e+03
mean      1.022000              5.615000  1.187295e+06
std       0.814973              2.887059  3.956232e+05
min       0.000000              1.000000  2.675197e+05
25%       0.000000              3.000000  8.731123e+05
50%       1.000000              6.000000  1.196792e+06
75%       2.000000              8.000000  1.503052e+06
max       2.000000             10.000000  2.084203e+06
```

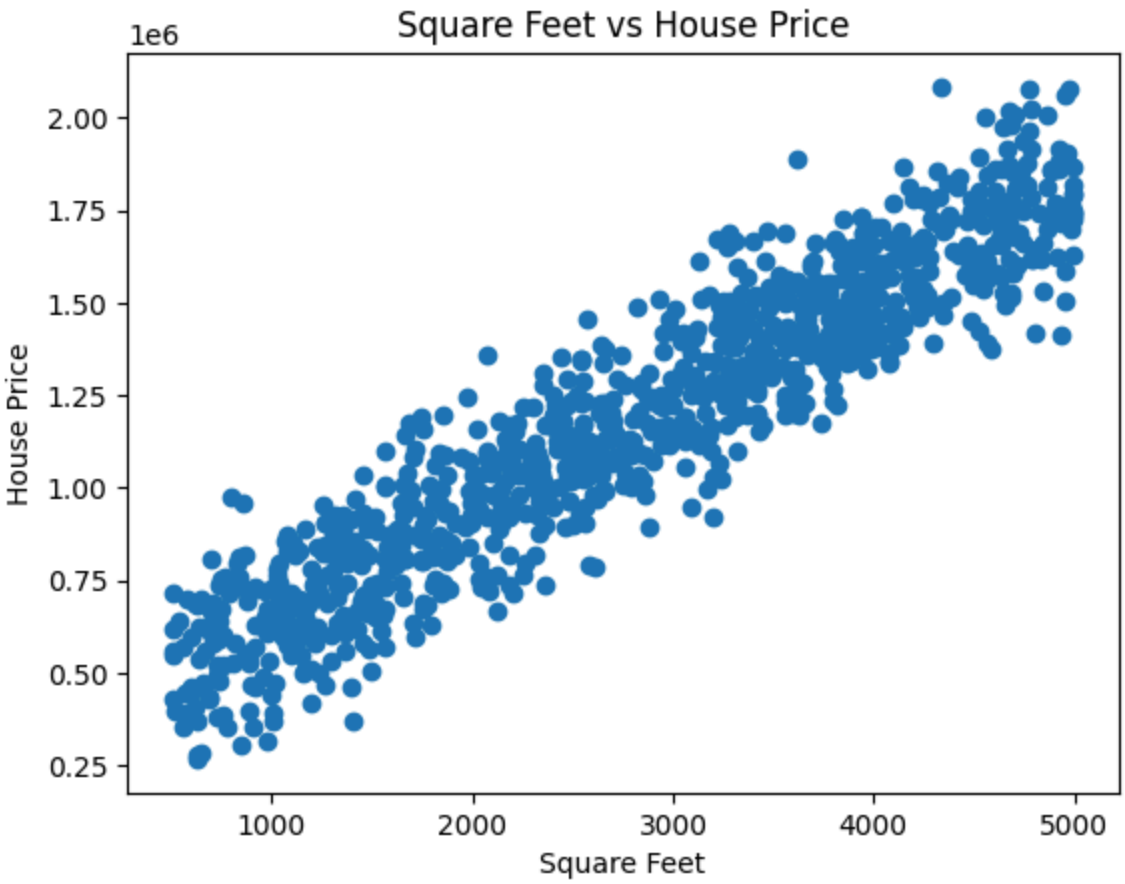
In [4]:
```
# ==============================
# Step 3: Data Preprocessing
# ==============================

# Check missing values
print("\nMissing Values:")
print(data.isnull().sum())

# Convert categorical variable 'Location' into numerical
data = pd.get_dummies(data, columns=['Location'], drop_first=True)

print("\nColumns after encoding:")
print(data.columns)
```

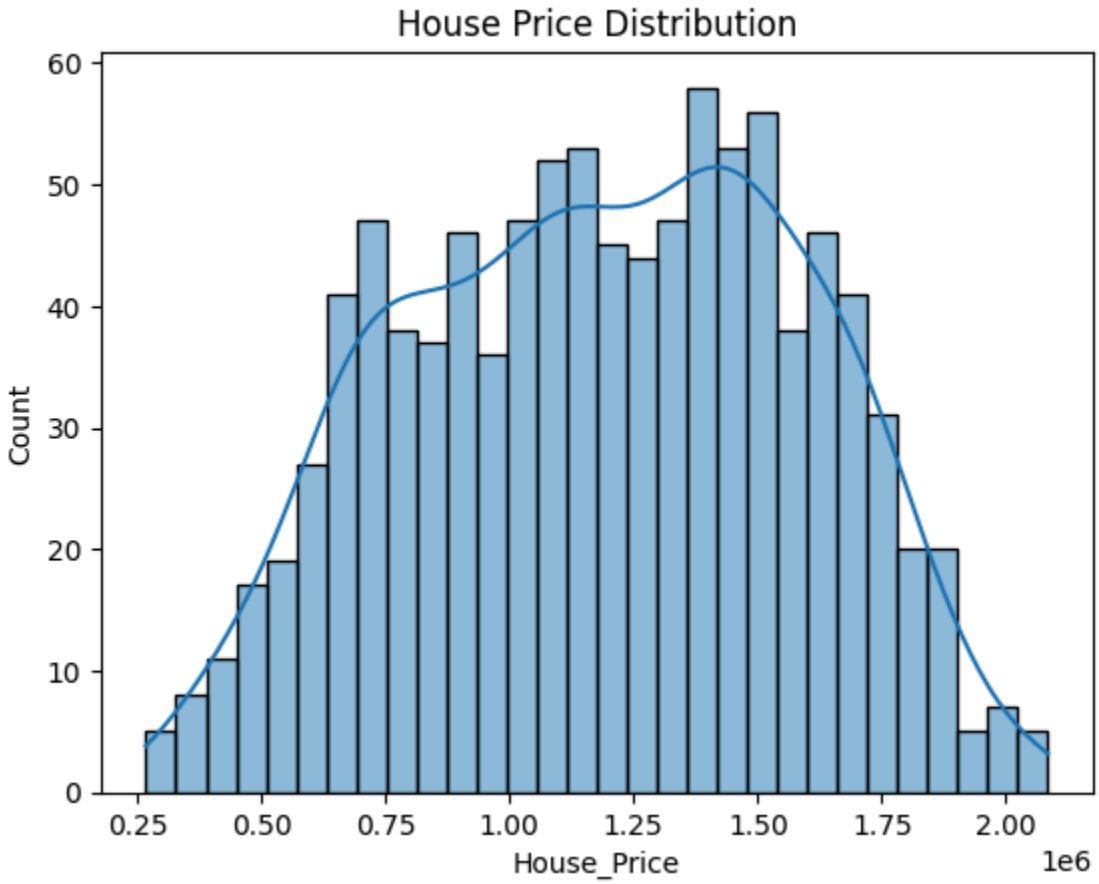
```
Missing Values:
Square_Feet             0
Num_Bedrooms            0
Num_Bathrooms           0
Year_Built              0
Lot_Size                0
Garage_Size             0
Neighborhood_Quality    0
Location                0
House_Price             0
dtype: int64

Columns after encoding:
Index(['Square_Feet', 'Num_Bedrooms', 'Num_Bathrooms', 'Year_Built',
       'Lot_Size', 'Garage_Size', 'Neighborhood_Quality', 'House_Price',
       'Location_chennai', 'Location_delhi', 'Location_mumbai',
       'Location_pune'],
      dtype='object')
```

In [5]:
```
# ==============================
# Step 4: Data Visualization
# ==============================

# Correlation heatmap
plt.figure(figsize=(10,7))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()

# Scatter plot: Size vs Price
plt.figure()
plt.scatter(data['Square_Feet'], data['House_Price'])
plt.xlabel("Square Feet")
plt.ylabel("House Price")
plt.title("Square Feet vs House Price")
plt.show()

# Distribution of house prices
plt.figure()
sns.histplot(data['House_Price'], bins=30, kde=True)
```

```python
plt.title("House Price Distribution")
plt.show()
```




Square Feet vs House Price
1e6
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
House Price
1000
2000
3000
4000
5000
Square Feet


House Price Distribution
60
50
40
30
20
10
0
Count
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
House_Price
1e6

In [6]:
```
# ==============================
# Step 5: Split Dataset
# ==============================

X = data.drop("House_Price", axis=1)
y = data["House_Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Size:", X_train.shape)
print("Testing Size:", X_test.shape)
```

```
Training Size: (800, 11)
Testing Size: (200, 11)
```

In [7]:
```
# ==============================
# Step 6: Train Linear Regression Model
# ==============================

model = LinearRegression()
model.fit(X_train, y_train)

print("\nModel Training Completed")
```

```
Model Training Completed
```

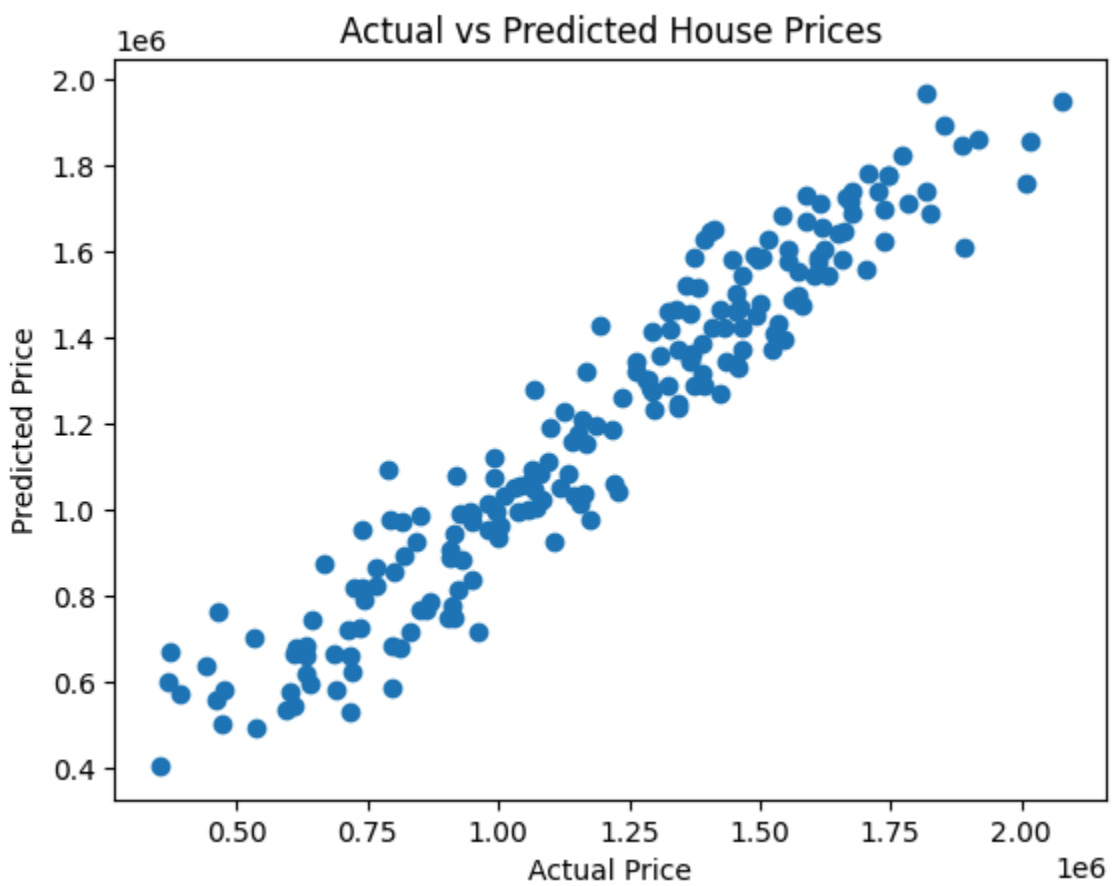
In [8]:
```
# ==============================
# Step 7: Prediction
# ==============================

y_pred = model.predict(X_test)

print("\nFirst 10 Predictions:")
print(y_pred[:10])
```

```
First 10 Predictions:
[1501699.29351249  971151.03335785 1756901.75930418 1653110.78018493
 1371892.90290653 1464039.43507711 1782822.08609159 1583872.17418435
 1461764.88070823 1546421.42692164]
```

In [9]:
```
# ==============================
# Step 8: Model Evaluation
# ==============================

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation")
print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R2 Score:", r2)
```

```
Model Evaluation
Mean Squared Error: 12159433288.896313
Root Mean Squared Error: 110269.82039024237
R2 Score: 0.921107101493902
```

In [10]:
```
# ==============================
# Step 9: Actual vs Predicted Plot
# ==============================

plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")
plt.show()
```



Slip 2

In [11]:
```
# Implementing the K-means clustering algorithm to identify distinct clusters
# a customer demographic dataset.
# a) Import the “Mall Customers” data and plot Visualizations.
# b) Carry out the required steps of data pre-processing.
# c) Install the model and fit it on the data according to the parameters.
# d) Show the final visualization of the clusters also find the number of
# clusters to be defined.
```

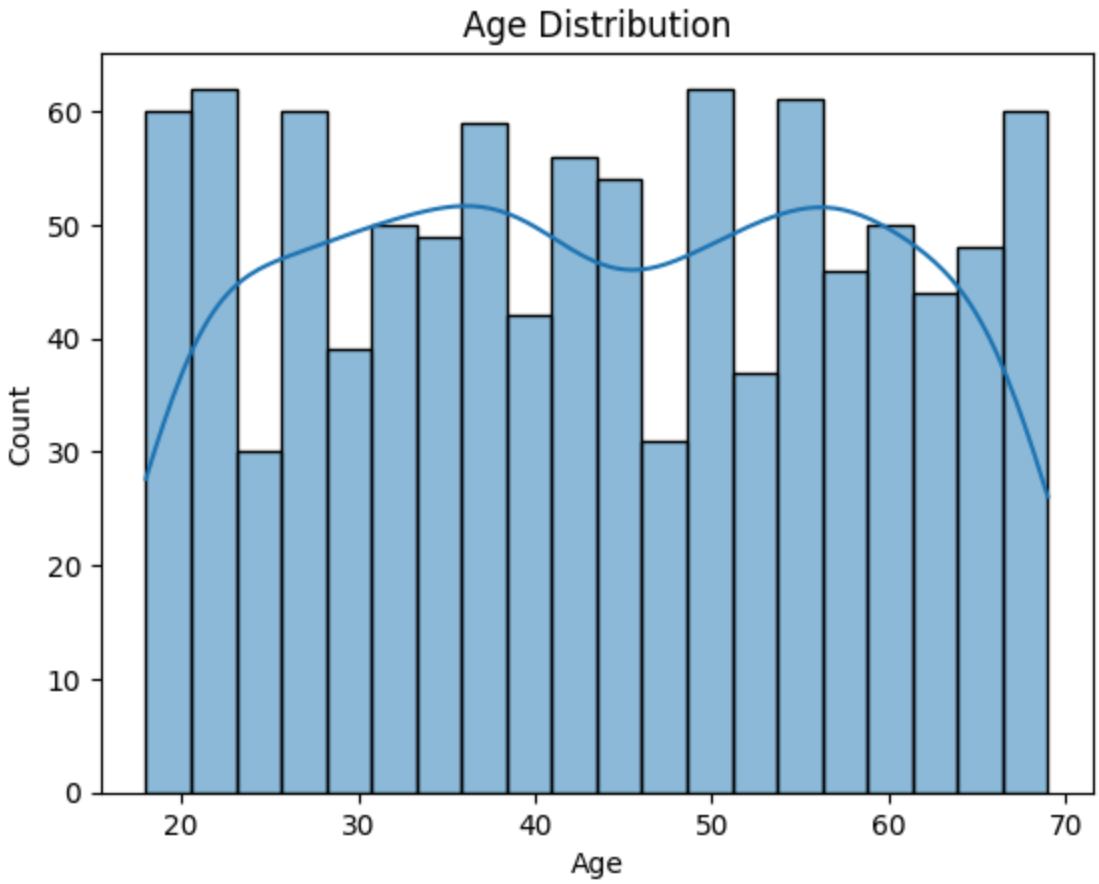
In [12]:
```python
# ==============================
# Step 1: Import Libraries
# ==============================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
```

In [13]:
```python
# ==============================
# Step 2: Import Dataset
# ==============================

data = pd.read_excel("clustering.xlsx")

print("First 5 rows:")
print(data.head())

print("\nDataset Info:")
print(data.info())

print("\nStatistical Summary:")
print(data.describe())
```

```
First 5 rows:
   CustomerID  AnnualIncome  SpendingScore  Age  Gender  MembershipYears  \
0           1        100569             11   26    Male               15
1           2         57220             51   25  Female               12
2           3         99062             83   19    Male               14
3           4         83251             64   36  Female                5
4           5         15063             33   58  Female                2

   PurchaseFrequency
0                 11
1                 37
2                  2
3                 15
4                 17

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         1000 non-null   int64
 1   AnnualIncome       1000 non-null   int64
 2   SpendingScore      1000 non-null   int64
 3   Age                1000 non-null   int64
 4   Gender             1000 non-null   object
 5   MembershipYears    1000 non-null   int64
 6   PurchaseFrequency  1000 non-null   int64
dtypes: int64(6), object(1)
memory usage: 54.8+ KB
None

Statistical Summary:
        CustomerID   AnnualIncome  SpendingScore         Age  MembershipYears
\
count  1000.000000    1000.000000    1000.000000  1000.00000      1000.000000
mean    500.500000   66088.510000      49.725000    43.40600        10.137000
std     288.819436   30445.776406      28.109705    14.97496         5.543551
min       1.000000   15063.000000       1.000000    18.00000         1.000000
25%     250.750000   38554.500000      26.000000    30.00000         5.000000
50%     500.500000   66141.000000      49.000000    43.00000        10.000000
75%     750.250000   93336.250000      73.000000    56.00000        15.000000
max    1000.000000  119898.000000      99.000000    69.00000        19.000000

       PurchaseFrequency
count         1000.00000
mean            25.17500
std             14.32395
min              1.00000
25%             13.00000
50%             26.00000
75%             38.00000
max             49.00000
```

In [14]:

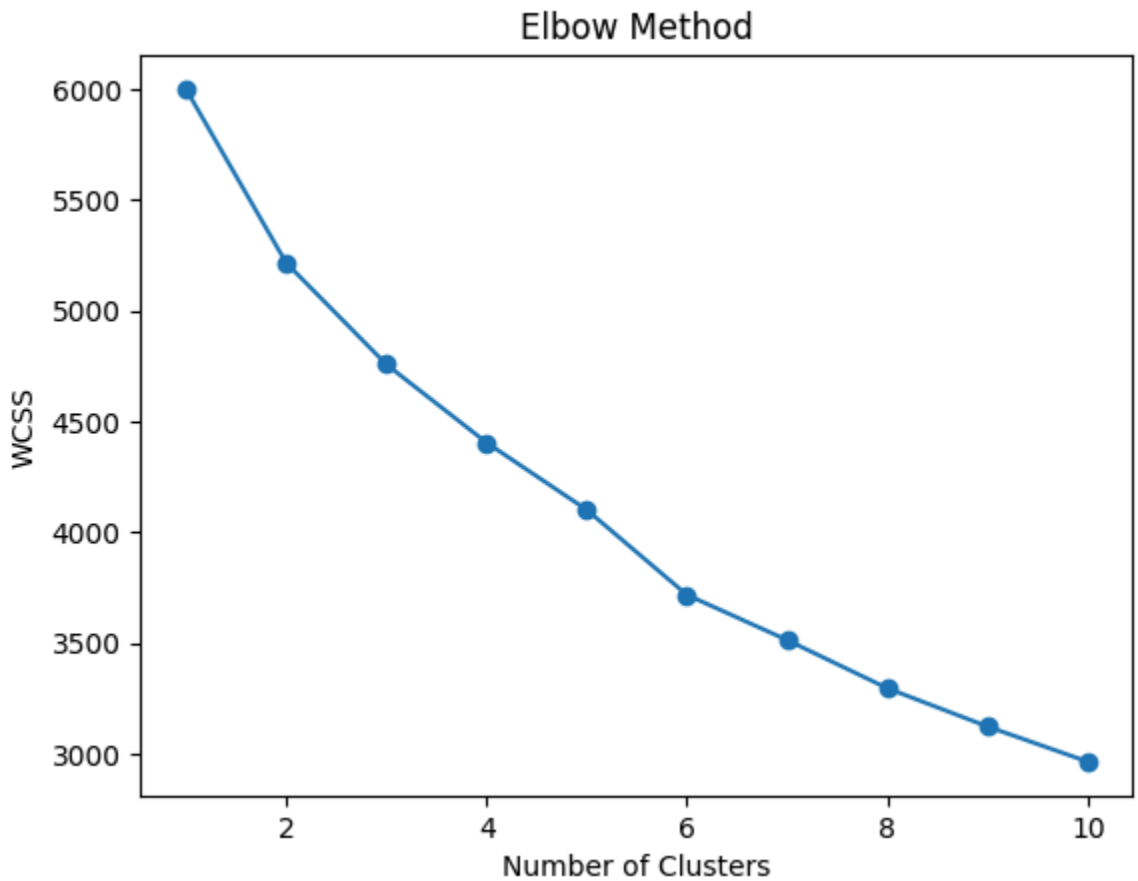
```
# ==============================
# Step 3: Data Visualization
# ==============================

# Income vs Spending Score
plt.figure()
plt.scatter(data["AnnualIncome"], data["SpendingScore"])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Income vs Spending Score")
plt.show()

# Age Distribution
plt.figure()
sns.histplot(data["Age"], bins=20, kde=True)
plt.title("Age Distribution")
plt.show()
```



In [15]:
```
# ==============================
# Step 4: Data Preprocessing
# ==============================

# Convert Gender to numeric
le = LabelEncoder()
data["Gender"] = le.fit_transform(data["Gender"])

# Remove CustomerID (not useful for clustering)
data = data.drop("CustomerID", axis=1)

# Feature Scaling
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

In [16]:
```
# ==============================
# Step 5: Find Optimal Number of Clusters (Elbow Method)
# ==============================

wcss = []

for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)
```

```
plt.figure()
plt.plot(range(1,11), wcss, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

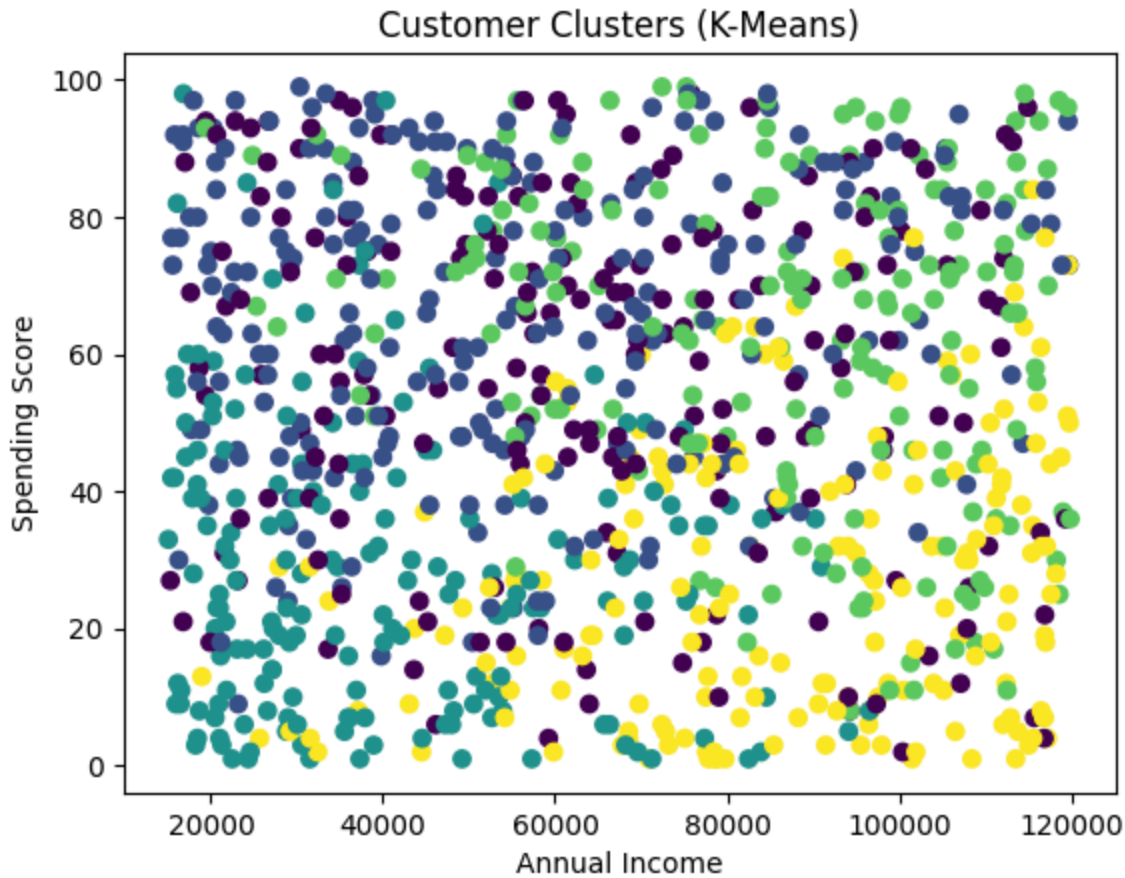


In [17]:
```
# ==============================
# Step 6: Train K-Means Model
# ==============================

kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(scaled_data)

data["Cluster"] = clusters

print("\nClustered Data:")
print(data.head())
```

```
Clustered Data:
   AnnualIncome  SpendingScore  Age  Gender  MembershipYears  \
0        100569             11   26       1               15
1         57220             51   25       0               12
2         99062             83   19       1               14
3         83251             64   36       0                5
4         15063             33   58       0                2

   PurchaseFrequency  Cluster
0                 11        4
1                 37        3
2                  2        1
3                 15        4
4                 17        2
```

In [18]:

```python
# ==============================
# Step 7: Final Cluster Visualization
# ==============================

plt.figure()

plt.scatter(data["AnnualIncome"], data["SpendingScore"],
            c=data["Cluster"])

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Clusters (K-Means)")

plt.show()
```



Slip 3

In [19]:
```
# Building a decision tree model using the ID3 algorithm to predict whether a
# customer will churn or not.
# a) Import the “Churn” data and analyse it with the help of pre-defined
# functions.
# b) Carry out the required steps of data pre-processing and do necessary
# Visualizations.
# c) Split the data and fit it with the Decision Tree model (use the model
# from “sklearn” library).
# d) Make the predictions and evaluate it on the test data.
```

In [20]:
```
# ==============================
# Step 1: Import Libraries
# ==============================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_m
```

In [21]:

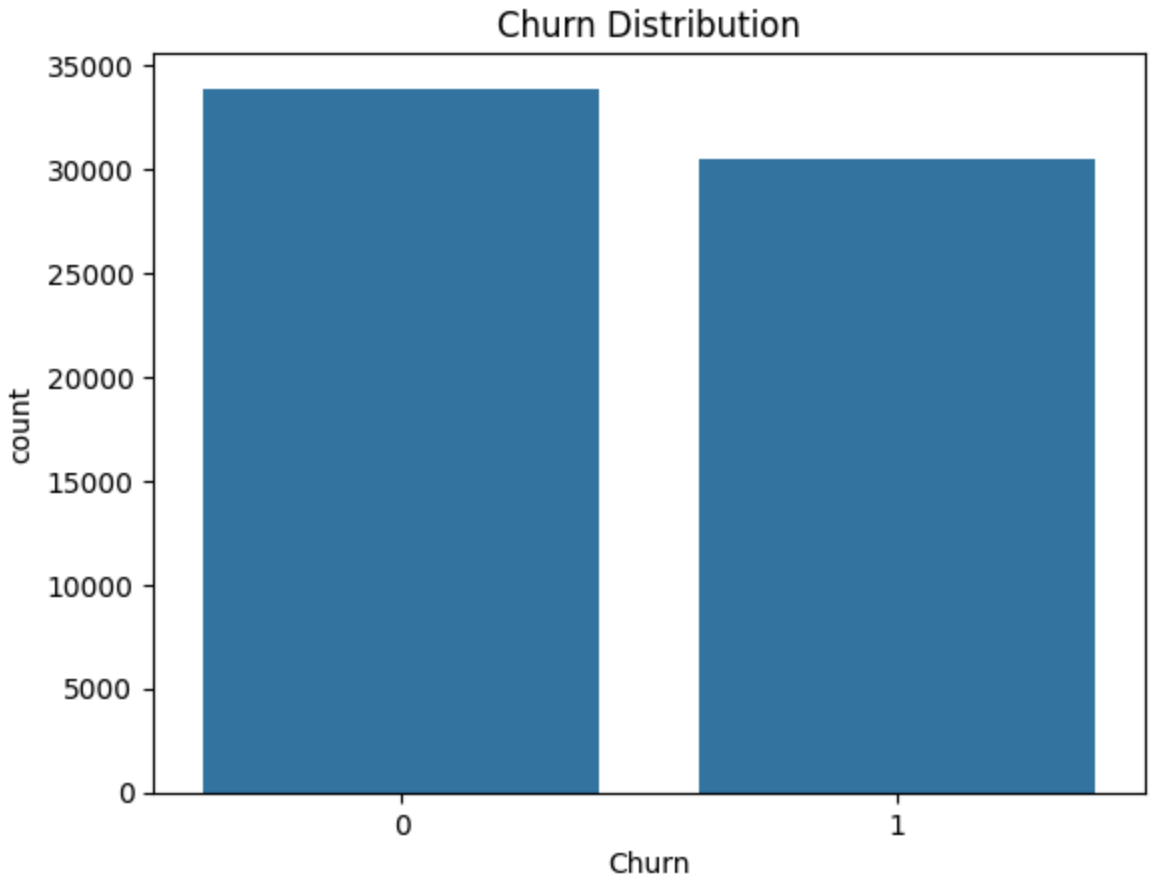
```python
# ==============================
# Step 2: Import Dataset
# ==============================

data = pd.read_csv("churn.csv")

print("First 5 Rows:")
print(data.head())

print("\nDataset Info:")
print(data.info())

print("\nStatistical Summary:")
print(data.describe())

print("\nMissing Values:")
print(data.isnull().sum())
```

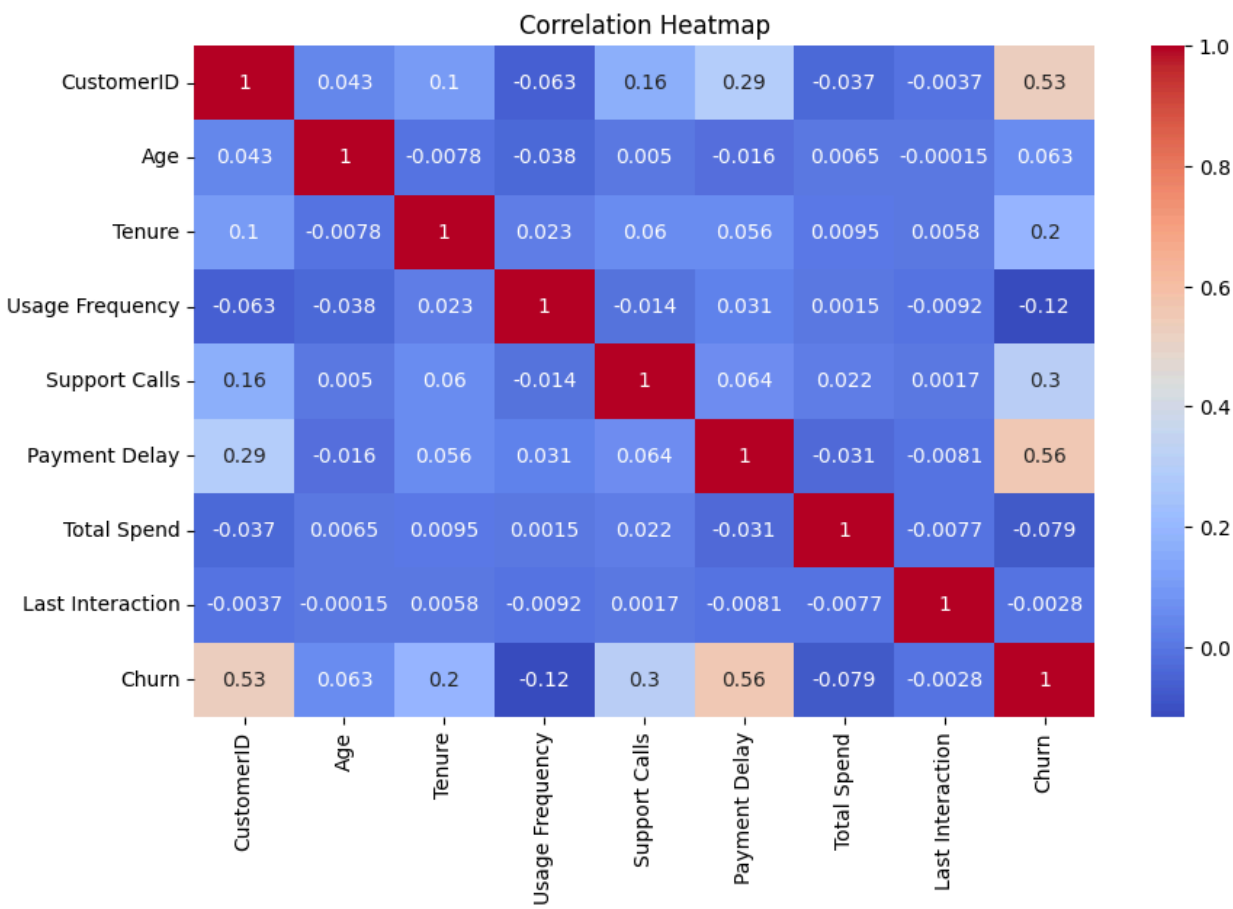
```
First 5 Rows:
   CustomerID  Age  Gender  Tenure  Usage Frequency  Support Calls  \
0           1   22  Female      25               14              4
1           2   41  Female      28               28              7
2           3   47    Male      27               10              2
3           4   35    Male       9               12              5
4           5   53  Female      58               24              9

   Payment Delay Subscription Type Contract Length  Total Spend  \
0             27             Basic         Monthly          598
1             13          Standard         Monthly          584
2             29           Premium          Annual          757
3             17           Premium       Quarterly          232
4              2          Standard          Annual          533

   Last Interaction  Churn
0                 9      1
1                20      0
2                21      0
3                18      0
4                18      0

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64374 entries, 0 to 64373
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         64374 non-null  int64
 1   Age                64374 non-null  int64
 2   Gender             64374 non-null  object
 3   Tenure             64374 non-null  int64
 4   Usage Frequency    64374 non-null  int64
 5   Support Calls      64374 non-null  int64
 6   Payment Delay      64374 non-null  int64
 7   Subscription Type  64374 non-null  object
 8   Contract Length    64374 non-null  object
 9   Total Spend        64374 non-null  int64
 10  Last Interaction   64374 non-null  int64
 11  Churn              64374 non-null  int64
dtypes: int64(9), object(3)
memory usage: 5.9+ MB
None

Statistical Summary:
         CustomerID           Age        Tenure  Usage Frequency  \
count  64374.000000  64374.000000  64374.000000     64374.000000
mean   32187.500000     41.970982     31.994827        15.080234
std    18583.317451     13.924911     17.098234         8.816470
min        1.000000     18.000000      1.000000         1.000000
25%    16094.250000     30.000000     18.000000         7.000000
50%    32187.500000     42.000000     33.000000        15.000000
75%    48280.750000     54.000000     47.000000        23.000000
max    64374.000000     65.000000     60.000000        30.000000
```

```
       Support Calls  Payment Delay   Total Spend  Last Interaction  \
count   64374.000000   64374.000000  64374.000000      64374.000000
mean        5.400690      17.133952    541.023379         15.498850
std         3.114005       8.852211    260.874809          8.638436
min         0.000000       0.000000    100.000000          1.000000
25%         3.000000      10.000000    313.000000          8.000000
50%         6.000000      19.000000    534.000000         15.000000
75%         8.000000      25.000000    768.000000         23.000000
max        10.000000      30.000000   1000.000000         30.000000

              Churn
count  64374.000000
mean       0.473685
std        0.499311
min        0.000000
25%        0.000000
50%        0.000000
75%        1.000000
max        1.000000

Missing Values:
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

In [22]:

```python
# ==============================
# Step 3: Data Visualization
# ==============================

# Churn distribution
plt.figure()
sns.countplot(x='Churn', data=data)
plt.title("Churn Distribution")
plt.show()

# Correlation Heatmap
plt.figure(figsize=(10,6))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```


Churn Distribution
count
35000
30000
25000
20000
15000
10000
5000
0
0
1
Churn


Correlation Heatmap
CustomerID 1 0.043 0.1 -0.063 0.16 0.29 -0.037 -0.0037 0.53
Age 0.043 1 -0.0078 -0.038 0.005 -0.016 0.0065 -0.00015 0.063
Tenure 0.1 -0.0078 1 0.023 0.06 0.056 0.0095 0.0058 0.2
Usage Frequency -0.063 -0.038 0.023 1 -0.014 0.031 0.0015 -0.0092 -0.12
Support Calls 0.16 0.005 0.06 -0.014 1 0.064 0.022 0.0017 0.3
Payment Delay 0.29 -0.016 0.056 0.031 0.064 1 -0.031 -0.0081 0.56
Total Spend -0.037 0.0065 0.0095 0.0015 0.022 -0.031 1 -0.0077 -0.079
Last Interaction -0.0037 -0.00015 0.0058 -0.0092 0.0017 -0.0081 -0.0077 1 -0.0028
Churn 0.53 0.063 0.2 -0.12 0.3 0.56 -0.079 -0.0028 1
CustomerID
Age
Tenure
Usage Frequency
Support Calls
Payment Delay
Total Spend
Last Interaction
Churn
1.0
0.8
0.6
0.4
0.2
0.0

In [23]:
```
# ==============================
# Step 4: Data Preprocessing
# ==============================

# Convert categorical columns to numeric
le = LabelEncoder()

for column in data.columns:
    if data[column].dtype == 'object':
        data[column] = le.fit_transform(data[column])

# Features and Target
X = data.drop("Churn", axis=1)
y = data["Churn"]
```

In [24]:
```
# ==============================
# Step 5: Split Dataset
# ==============================

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (51499, 11)
Testing Data Shape: (12875, 11)
```

In [25]:
```
# ==============================
# Step 6: Train Decision Tree Model (ID3)
# ==============================

model = DecisionTreeClassifier(criterion="entropy", random_state=42)

model.fit(X_train, y_train)

print("\nModel Training Completed")
```

```
Model Training Completed
```

In [26]:
```
# ==============================
# Step 7: Prediction
# ==============================

y_pred = model.predict(X_test)

print("\nFirst 10 Predictions:")
print(y_pred[:10])
```

```
First 10 Predictions:
[0 0 1 0 0 0 0 0 1 0]
```

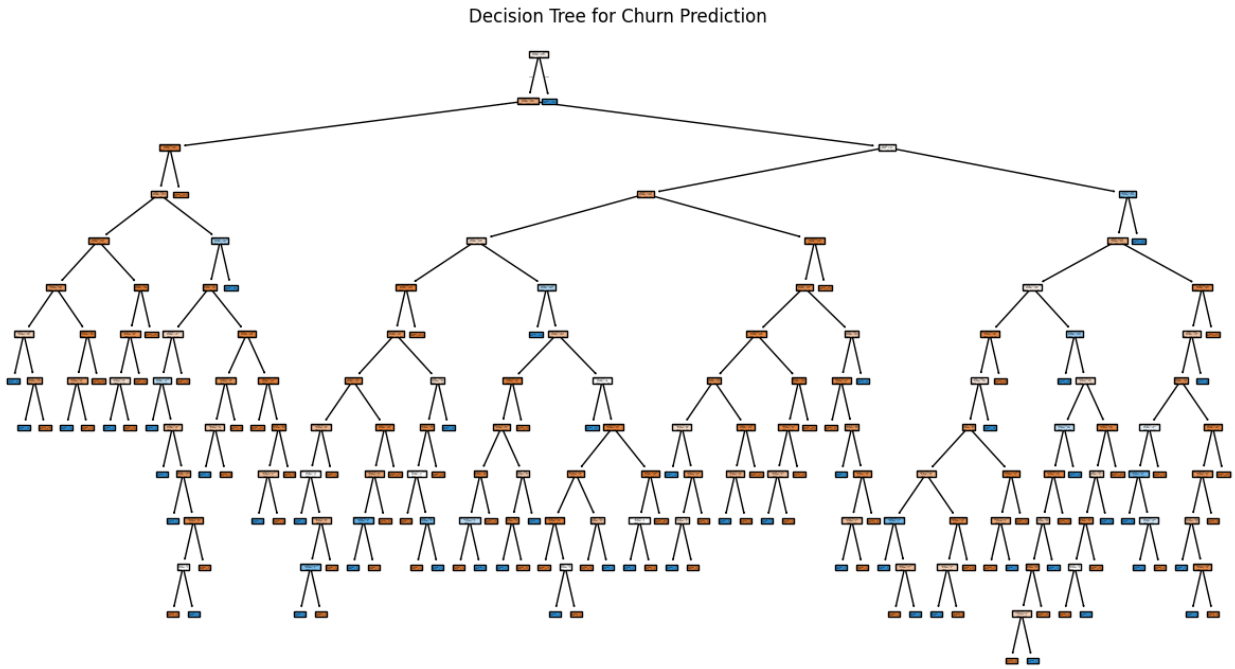
In [27]:
```
# ==============================
```

```python
# Step 8: Model Evaluation
# ==============================

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Model Accuracy: 0.9986796116504855

Confusion Matrix:
[[6784    9]
 [   8 6074]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      6793
           1       1.00      1.00      1.00      6082

    accuracy                           1.00     12875
   macro avg       1.00      1.00      1.00     12875
weighted avg       1.00      1.00      1.00     12875
```

In [28]:

```python
# ==============================
# Step 9: Decision Tree Visualization
# ==============================

from sklearn.tree import plot_tree

plt.figure(figsize=(15,8))
plot_tree(model, filled=True, feature_names=X.columns)
plt.title("Decision Tree for Churn Prediction")
plt.show()
```



Slip 4

In [29]:
```
# Using logistic regression predict whether a credit card transaction is
# fraudulent or not.
# a) Import the “Credit Card” data and remove the variables not required
# for the processing.
# b) Split the data into training and validation data.
# c) Train the data on Logistic Regression model and predict for the test
# data.
# d) Evaluate the predictions i.e. show the Confusion Matrix and Accuracy
# Score of the model.
```

In [30]:
```
# ==============================
# Step 1: Import Libraries
# ==============================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score
```

In [31]:
```
# ==============================
# Step 2: Import Dataset
# ==============================

data = pd.read_csv("creditcard.csv")
```

```python
print("First 5 rows:")
print(data.head())

print("\nDataset Info:")
print(data.info())

print("\nClass Distribution:")
print(data["Class"].value_counts())
```

```
First 5 rows:
     Time        V1        V2        V3        V4        V5        V6  \
0  121958  0.486036  0.101723 -0.815592 -1.721047 -0.002736 -0.843590
1  146867 -1.547304  1.498012 -0.349863 -0.982416 -0.394599 -0.474763
2  131932  1.082691 -0.312836  0.649581 -0.430415  0.396706  1.254656
3  103694 -0.471125  1.030919 -0.342718  1.368189 -0.193658  1.796147
4  119879 -0.093636 -0.288063  1.078426  0.901144 -0.085176 -1.264686

         V7        V8        V9  ...       V21       V22       V23       V24  \
0  0.354177 -0.545922  1.117118  ...  0.088659  0.809815  0.139065  0.675980
1  0.483706 -1.404541  0.599804  ... -2.018395  1.165325 -1.998137 -0.797603
2  0.227148 -0.893522 -1.276280  ...  0.299506  0.014265 -0.239139 -1.432304
3 -0.616184 -0.640656  0.447960  ...  0.610152 -0.232129  1.306864  0.490955
4 -0.201492 -0.196552  1.047670  ... -0.459552 -0.070654 -1.066149  0.447087

        V25       V26       V27       V28  Amount  Class
0  1.365122 -1.952076  0.139163 -1.452492  157.89      0
1 -0.028983 -0.389299 -0.235003 -0.179763    3.77      0
2  1.484852  0.707765  2.008292 -1.269684  251.68      0
3  0.839080 -0.166274  1.483266 -0.477960    3.94      0
4 -0.418308  0.173153 -0.608651 -2.035171    1.70      0

[5 rows x 31 columns]

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 31 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Time    5000 non-null   int64
 1   V1      5000 non-null   float64
 2   V2      5000 non-null   float64
 3   V3      5000 non-null   float64
 4   V4      5000 non-null   float64
 5   V5      5000 non-null   float64
 6   V6      5000 non-null   float64
 7   V7      5000 non-null   float64
 8   V8      5000 non-null   float64
 9   V9      5000 non-null   float64
 10  V10     5000 non-null   float64
 11  V11     5000 non-null   float64
 12  V12     5000 non-null   float64
 13  V13     5000 non-null   float64
 14  V14     5000 non-null   float64
 15  V15     5000 non-null   float64
 16  V16     5000 non-null   float64
 17  V17     5000 non-null   float64
 18  V18     5000 non-null   float64
 19  V19     5000 non-null   float64
 20  V20     5000 non-null   float64
 21  V21     5000 non-null   float64
 22  V22     5000 non-null   float64
 23  V23     5000 non-null   float64
```

```
 24  V24     5000 non-null   float64
 25  V25     5000 non-null   float64
 26  V26     5000 non-null   float64
 27  V27     5000 non-null   float64
 28  V28     5000 non-null   float64
 29  Amount  5000 non-null   float64
 30  Class   5000 non-null   int64
dtypes: float64(29), int64(2)
memory usage: 1.2 MB
None

Class Distribution:
Class
0    4900
1     100
Name: count, dtype: int64
```

In [32]:

```
# ==============================
# Step 3: Remove Unnecessary Variables
# ==============================

# Time column usually not required
data = data.drop("Time", axis=1)
```

In [33]:

```
# ==============================
# Step 4: Data Visualization
# ==============================

plt.figure()
sns.countplot(x="Class", data=data)
plt.title("Fraud vs Normal Transactions")
plt.show()
```

In [34]:
```
# ==============================
# Step 5: Define Features and Target
# ==============================

X = data.drop("Class", axis=1)
y = data["Class"]
```

In [35]:
```
# ==============================
# Step 6: Split Dataset
# ==============================

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (4000, 29)
Testing Shape: (1000, 29)
```

In [36]:
```
# ==============================
# Step 7: Train Logistic Regression Model
# ==============================
```

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

print("\nModel Training Completed")
```

```
Model Training Completed
```

In [37]:

```
# ==============================
# Step 8: Predictions
# ==============================

y_pred = model.predict(X_test)

print("\nFirst 10 Predictions:")
print(y_pred[:10])
```

```
First 10 Predictions:
[0 0 0 0 0 0 0 0 0 0]
```

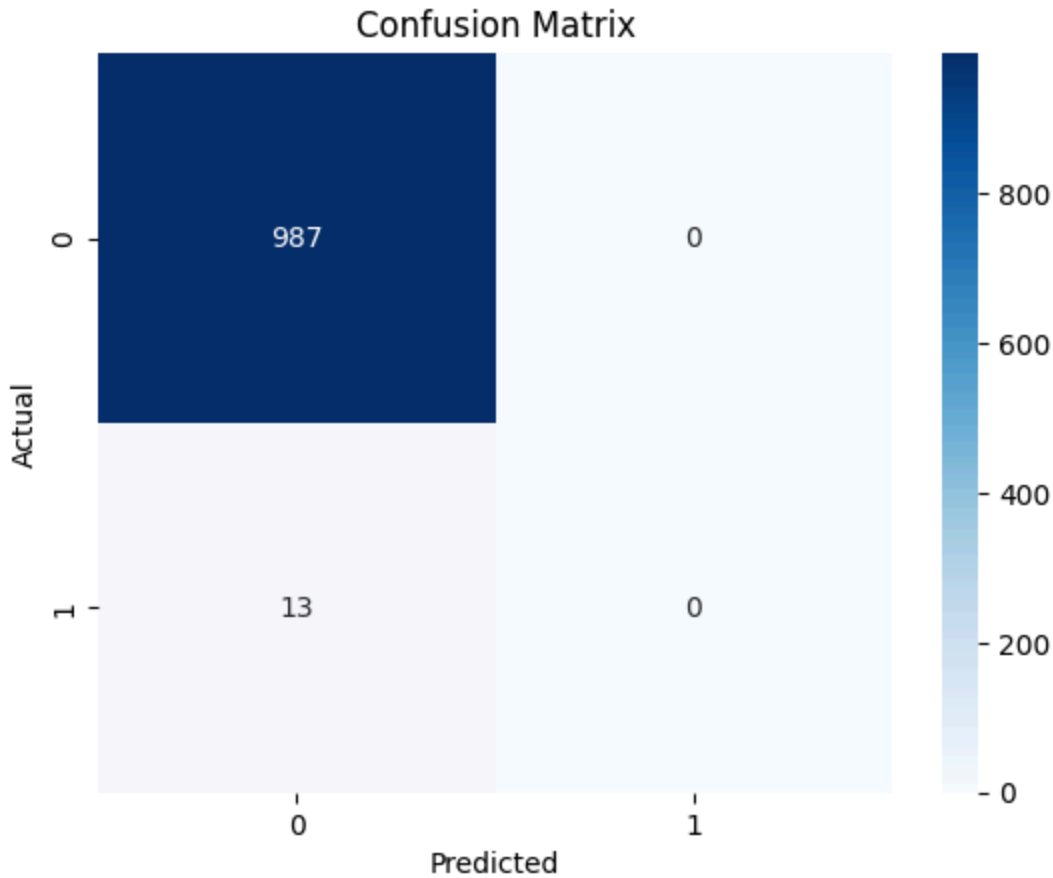
In [38]:

```
# ==============================
# Step 9: Model Evaluation
# ==============================

cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

print("\nAccuracy Score:", accuracy)
```

```
Confusion Matrix:
[[987   0]
 [ 13   0]]

Accuracy Score: 0.987
```

In [39]:

```
# ==============================
# Step 10: Confusion Matrix Visualization
# ==============================

plt.figure()
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Slip 5

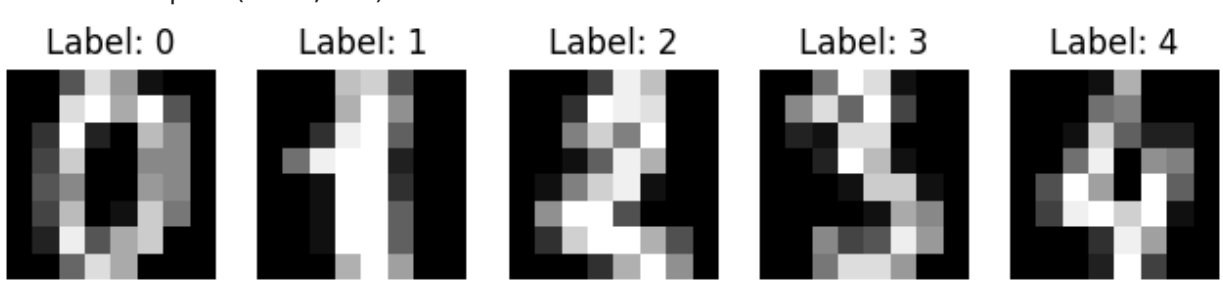
In [40]:
```
# Implementing a K-Nearest Neighbor (KNN) algorithm to classify handwritten
# digits.
# a) Import the “Digits Classification” data and plot a figure to know the
# data.
# b) Split the data into training and validation data also define the main
# “KNN” function.
# c) Create the Euclidean and Accuracy Score functions and evaluate the
# KNN model.
# d) Predict the test data and make a for loop to show few values and label
# the model that are predicted.
```

In [41]:
```
# ==============================
# Step 1: Import Libraries
# ==============================
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# ==============================
```

```python
# Step 2: Import Digits Dataset
# ==============================

digits = load_digits()

X = digits.data
y = digits.target

print("Dataset Shape:", X.shape)

# Plot few digit images
plt.figure(figsize=(8,4))

for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(digits.images[i], cmap='gray')
    plt.title("Label: " + str(y[i]))
    plt.axis('off')

plt.show()

# ==============================
# Step 3: Split Dataset
# ==============================

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

# ==============================
# Step 4: Euclidean Distance Function
# ==============================

def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

# ==============================
# Step 5: KNN Function
# ==============================

def knn_predict(X_train, y_train, x_test, k=3):

    distances = []

    # Calculate distance from test point to all training points
    for i in range(len(X_train)):
        dist = euclidean_distance(x_test, X_train[i])
        distances.append((dist, y_train[i]))

    # Sort distances
```

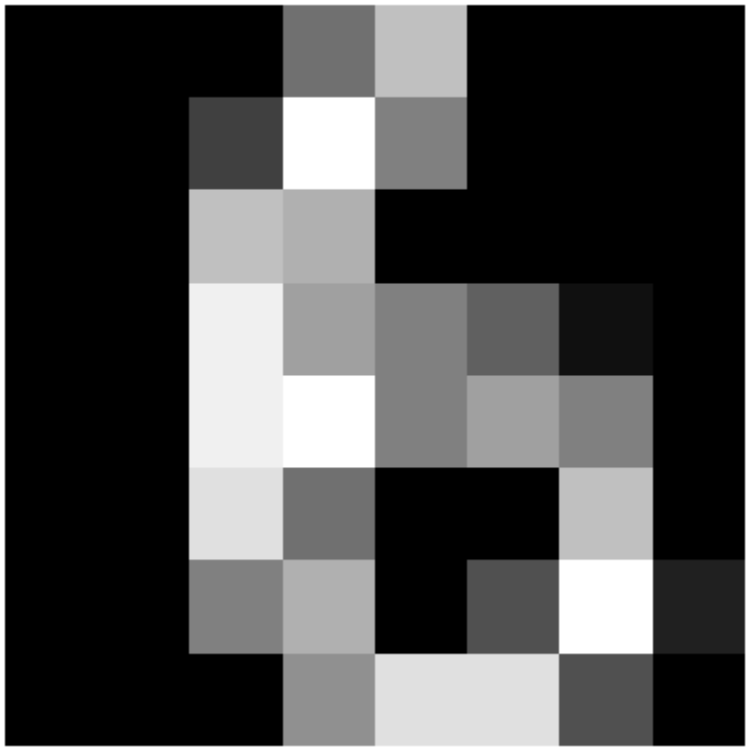
```python
    distances.sort(key=lambda x: x[0])

    # Take k nearest neighbors
    neighbors = distances[:k]

    labels = [label for (_, label) in neighbors]

    # Majority voting
    prediction = max(set(labels), key=labels.count)

    return prediction

# ==============================
# Step 6: Predict Test Data
# ==============================

predictions = []

for x in X_test:
    pred = knn_predict(X_train, y_train, x, k=3)
    predictions.append(pred)

predictions = np.array(predictions)

# ==============================
# Step 7: Evaluate Model
# ==============================

accuracy = accuracy_score(y_test, predictions)

print("Model Accuracy:", accuracy)

# ==============================
# Step 8: Show Few Predictions
# ==============================

for i in range(10):

    plt.imshow(X_test[i].reshape(8,8), cmap='gray')
    plt.title("Predicted: " + str(predictions[i]))
    plt.axis('off')
    plt.show()
```

```
Dataset Shape: (1797, 64)
```

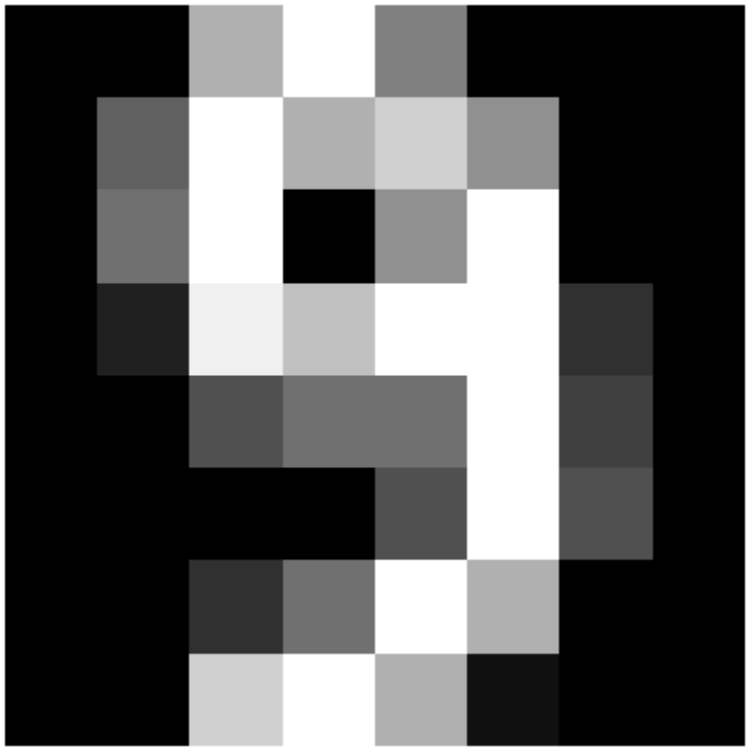


```
Training samples: 1437
Testing samples: 360
Model Accuracy: 0.9833333333333333
```

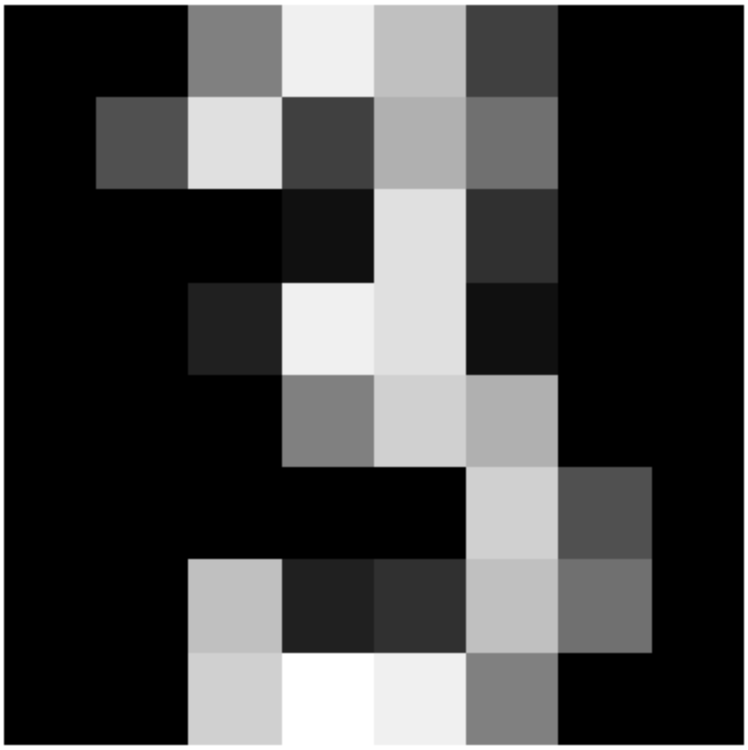
Predicted: 6



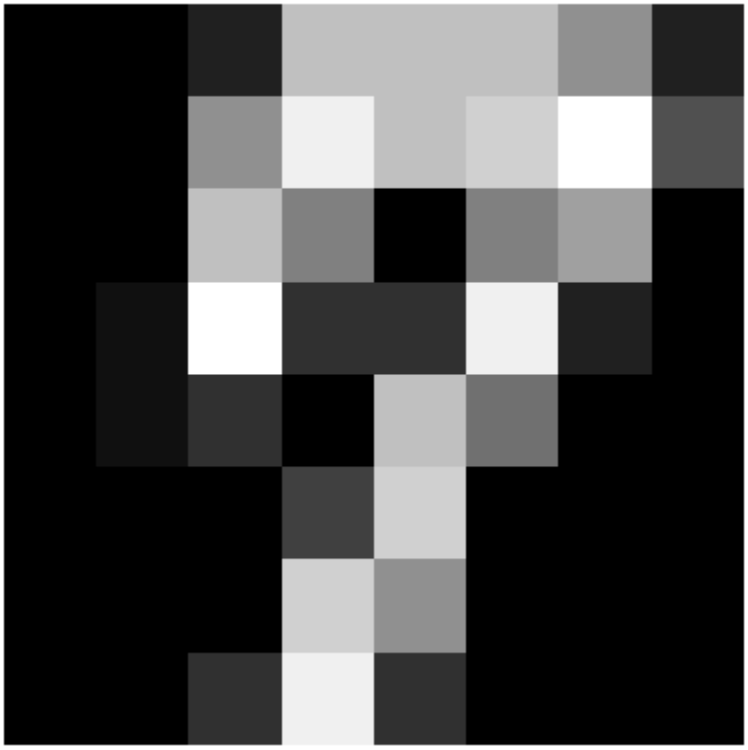
Predicted: 9



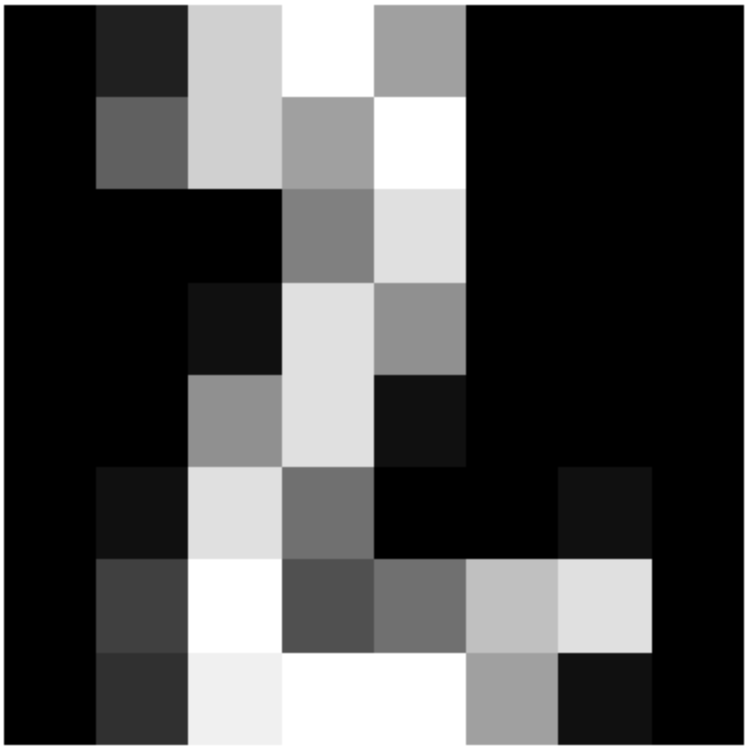
Predicted: 3



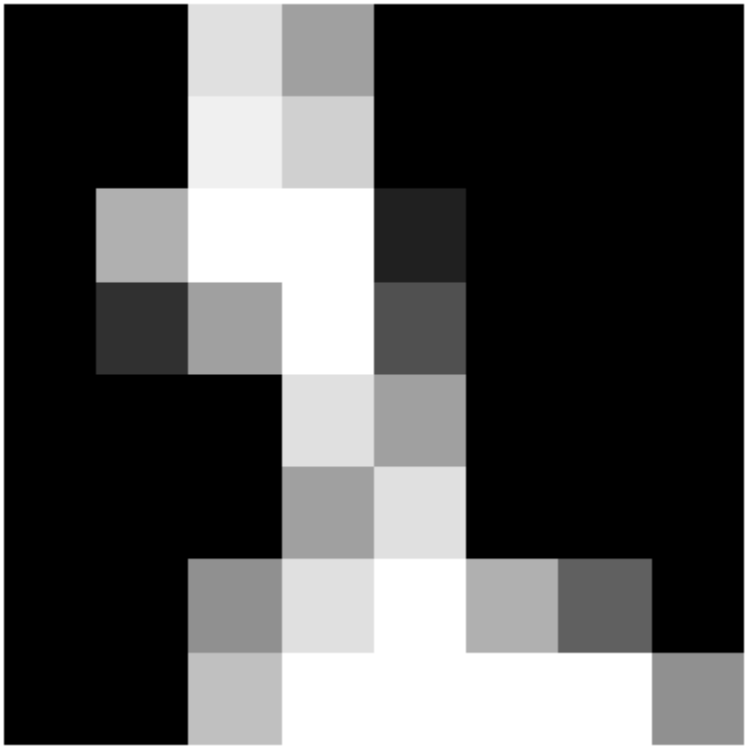
Predicted: 7



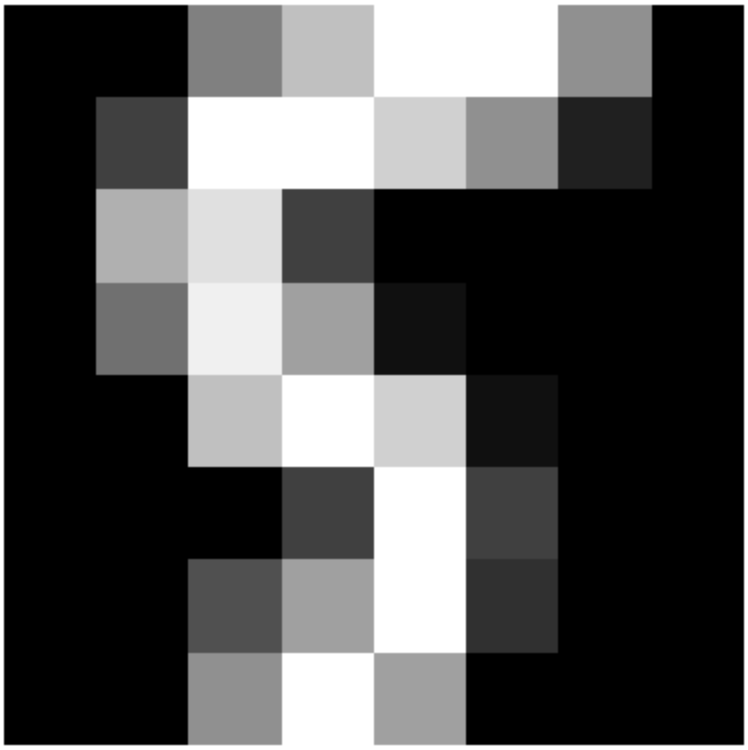
Predicted: 2



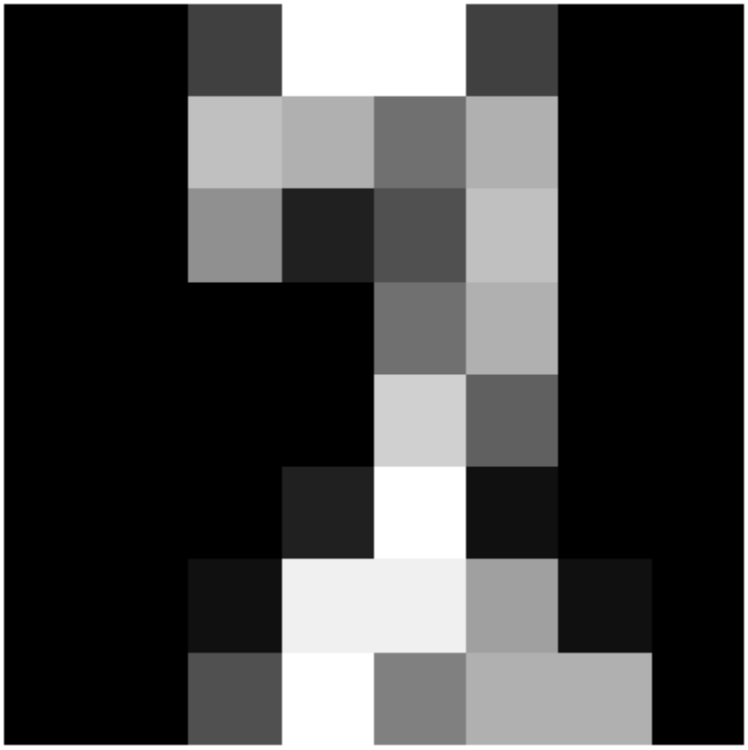
Predicted: 1



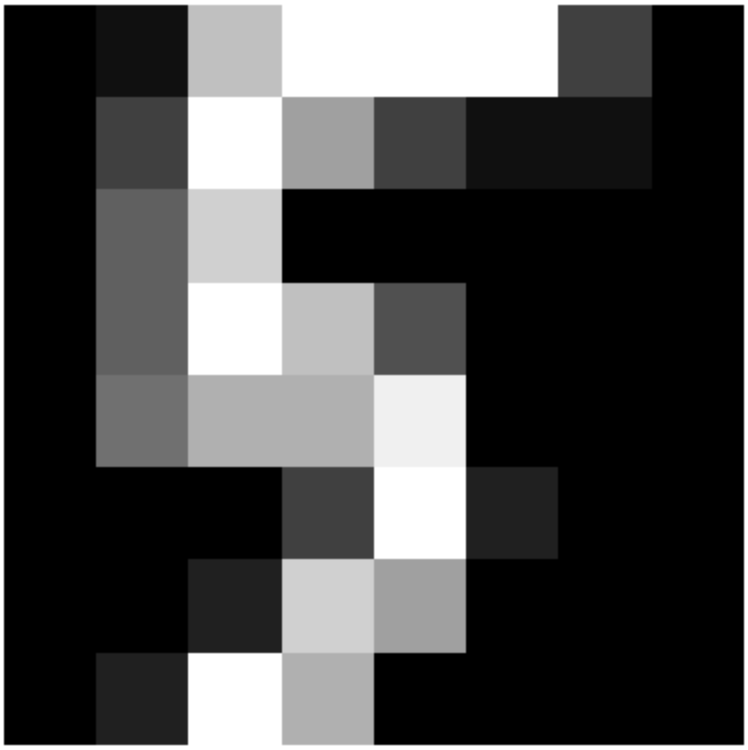
Predicted: 5



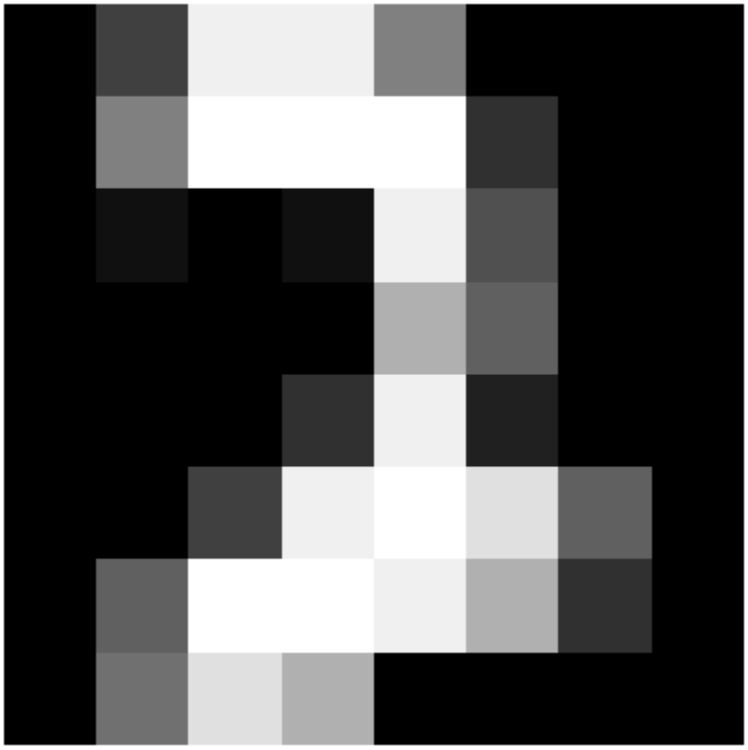
Predicted: 2



Predicted: 5



Predicted: 2



Slip 6

In [42]:
```
# Applying hierarchical clustering to group customer segments based on their
# purchasing behavior.
# a) Import the “Mall Customers” data and plot Visualizations.
# b) Carry out the required steps of data pre-processing.
# c) Install the model and fit it on the data according to the parameters.
# d) Show the final visualization of the dendogram by defining the function
# for the same.
```

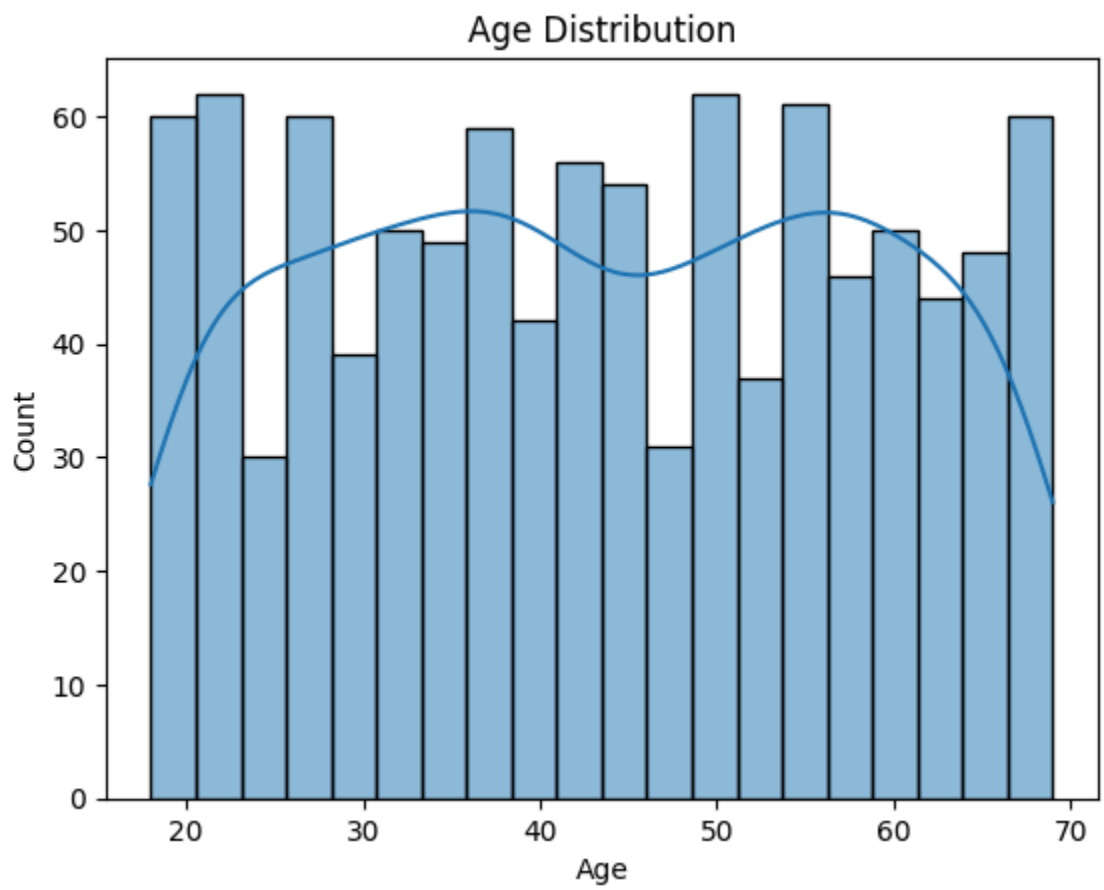
In [43]:
```
# ==============================
# Step 1: Import Libraries
# ==============================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering

from scipy.cluster.hierarchy import dendrogram, linkage


# ==============================
# Step 2: Import Dataset
# ==============================

data = pd.read_excel("clustering.xlsx")

print("First 5 Rows")
print(data.head())

print("\nDataset Info")
print(data.info())

print("\nStatistical Summary")
print(data.describe())


# ==============================
# Step 3: Data Visualization
# ==============================

# Income vs Spending Score
plt.figure()
plt.scatter(data["AnnualIncome"], data["SpendingScore"])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Income vs Spending Score")
```

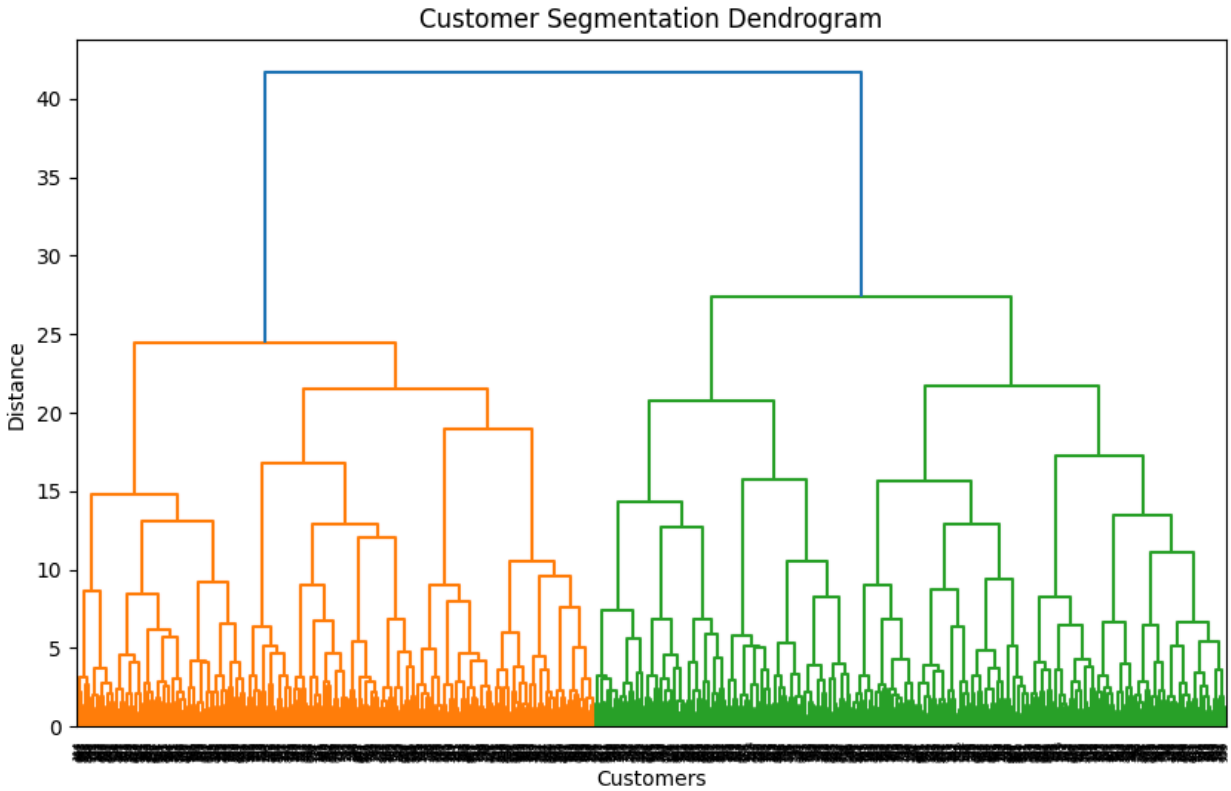
```python
plt.show()

# Age distribution
plt.figure()
sns.histplot(data["Age"], bins=20, kde=True)
plt.title("Age Distribution")
plt.show()


# ==============================
# Step 4: Data Preprocessing
# ==============================

# Convert Gender to numeric
le = LabelEncoder()
data["Gender"] = le.fit_transform(data["Gender"])

# Remove CustomerID
data = data.drop("CustomerID", axis=1)

# Feature Scaling
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)


# ==============================
# Step 5: Dendrogram Function
# ==============================

def plot_dendrogram(data):

    linked = linkage(data, method='ward')

    plt.figure(figsize=(10,6))
    dendrogram(linked)

    plt.title("Customer Segmentation Dendrogram")
    plt.xlabel("Customers")
    plt.ylabel("Distance")

    plt.show()

# Call dendrogram function
plot_dendrogram(scaled_data)


# ==============================
# Step 6: Fit Hierarchical Model
# ==============================

model = AgglomerativeClustering(n_clusters=5)

clusters = model.fit_predict(scaled_data)
```

```python
data["Cluster"] = clusters

print("\nClustered Data")
print(data.head())


# ==============================
# Step 7: Final Cluster Visualization
# ==============================

plt.figure()

plt.scatter(data["AnnualIncome"], data["SpendingScore"],
            c=data["Cluster"])

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Segments (Hierarchical Clustering)")

plt.show()
```

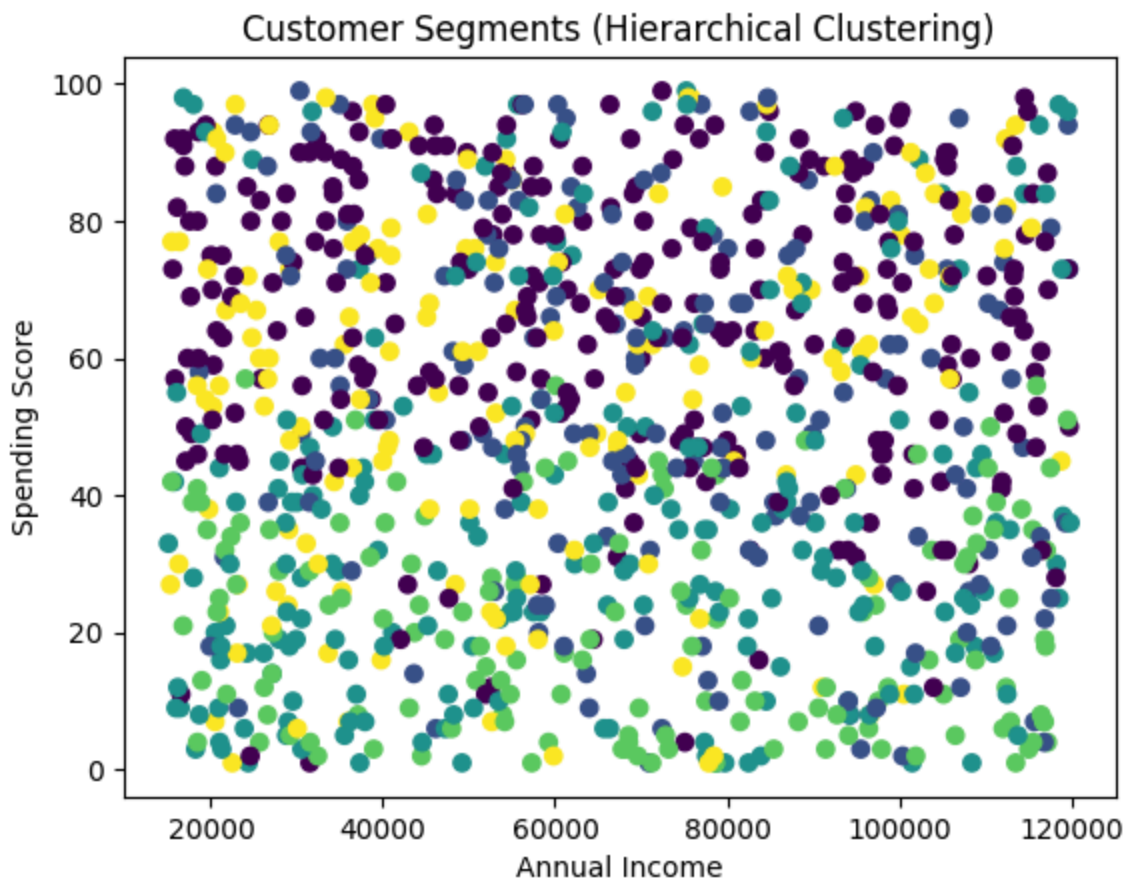
```
First 5 Rows
   CustomerID  AnnualIncome  SpendingScore  Age  Gender  MembershipYears  \
0           1        100569             11   26    Male               15
1           2         57220             51   25  Female               12
2           3         99062             83   19    Male               14
3           4         83251             64   36  Female                5
4           5         15063             33   58  Female                2

   PurchaseFrequency
0                 11
1                 37
2                  2
3                 15
4                 17

Dataset Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         1000 non-null   int64
 1   AnnualIncome       1000 non-null   int64
 2   SpendingScore      1000 non-null   int64
 3   Age                1000 non-null   int64
 4   Gender             1000 non-null   object
 5   MembershipYears    1000 non-null   int64
 6   PurchaseFrequency  1000 non-null   int64
dtypes: int64(6), object(1)
memory usage: 54.8+ KB
None

Statistical Summary
        CustomerID   AnnualIncome  SpendingScore         Age  MembershipYears
\
count  1000.000000    1000.000000    1000.000000  1000.00000      1000.000000
mean    500.500000   66088.510000      49.725000    43.40600        10.137000
std     288.819436   30445.776406      28.109705    14.97496         5.543551
min       1.000000   15063.000000       1.000000    18.00000         1.000000
25%     250.750000   38554.500000      26.000000    30.00000         5.000000
50%     500.500000   66141.000000      49.000000    43.00000        10.000000
75%     750.250000   93336.250000      73.000000    56.00000        15.000000
max    1000.000000  119898.000000      99.000000    69.00000        19.000000

       PurchaseFrequency
count         1000.00000
mean            25.17500
std             14.32395
min              1.00000
25%             13.00000
50%             26.00000
75%             38.00000
max             49.00000
```

Income vs Spending Score
Spending Score
Annual Income
0
20
40
60
80
100
20000
40000
60000
80000
100000
120000


Age Distribution
Count
Age
0
10
20
30
40
50
60
20
30
40
50
60
70




```
Clustered Data
   AnnualIncome  SpendingScore  Age  Gender  MembershipYears  \
0        100569             11   26       1               15
1         57220             51   25       0               12
2         99062             83   19       1               14
3         83251             64   36       0                5
4         15063             33   58       0                2

   PurchaseFrequency  Cluster
0                 11        4
1                 37        0
2                  2        4
3                 15        0
4                 17        2
```



Slip 7

In [44]:
```
# Developing a Support Vector Machine (SVM) model to classify email
# messages as spam or not spam.
# a) Import the “Spam Classification” data and also install the necessary
# libraries related to NLP.
# b) Define the necessary pre-processing functions and delete the
# duplicates if any.
# c) Split the data into training and testing, also extract important features
# d) Train the data on SVM algorithm and find out the Accuracy Score and
# Confusion Matrix by making predictions on the test data.
```

In [45]:
```
# ==============================
# Step 1: Import Libraries
# ==============================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# NLP libraries
import re
import nltk
```

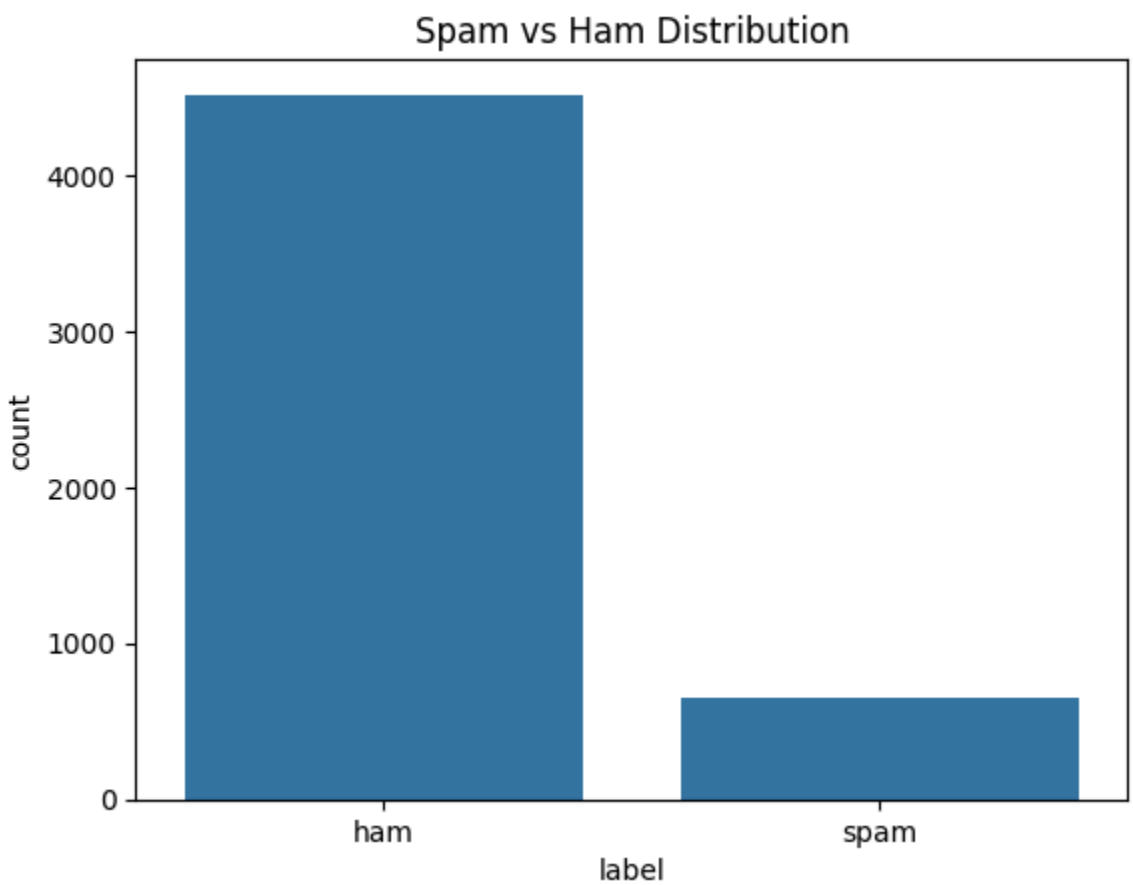
```python
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Feature extraction
from sklearn.feature_extraction.text import TfidfVectorizer

# Model
from sklearn.svm import SVC

# Train/Test split
from sklearn.model_selection import train_test_split

# Evaluation
from sklearn.metrics import accuracy_score, confusion_matrix


# Download stopwords
nltk.download('stopwords')


# ==============================
# Step 2: Import Dataset
# ==============================

data = pd.read_csv("spam.csv", encoding="latin-1")

print("First 5 rows:")
print(data.head())

print("\nDataset Info:")
print(data.info())


# Keep only required columns
data = data[['label','message']]


# ==============================
# Step 3: Remove Duplicates
# ==============================

data = data.drop_duplicates()

print("\nDataset Shape After Removing Duplicates:", data.shape)


# ==============================
# Step 4: Data Visualization
# ==============================

plt.figure()
sns.countplot(x='label', data=data)
plt.title("Spam vs Ham Distribution")
```

```python
plt.show()


# ==============================
# Step 5: Text Preprocessing Function
# ==============================

ps = PorterStemmer()

def preprocess(text):

    text = text.lower()
    text = re.sub('[^a-zA-Z]', ' ', text)

    words = text.split()

    words = [ps.stem(word) for word in words if word not in stopwords.words('e

    return " ".join(words)


# Apply preprocessing
data['clean_message'] = data['message'].apply(preprocess)


# ==============================
# Step 6: Convert Text to Features
# ==============================

tfidf = TfidfVectorizer(max_features=3000)

X = tfidf.fit_transform(data['clean_message']).toarray()

# Convert labels to numeric
y = data['label'].map({'ham':0,'spam':1})


# ==============================
# Step 7: Train-Test Split
# ==============================

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)


# ==============================
# Step 8: Train SVM Model
# ==============================

model = SVC(kernel='linear')
```

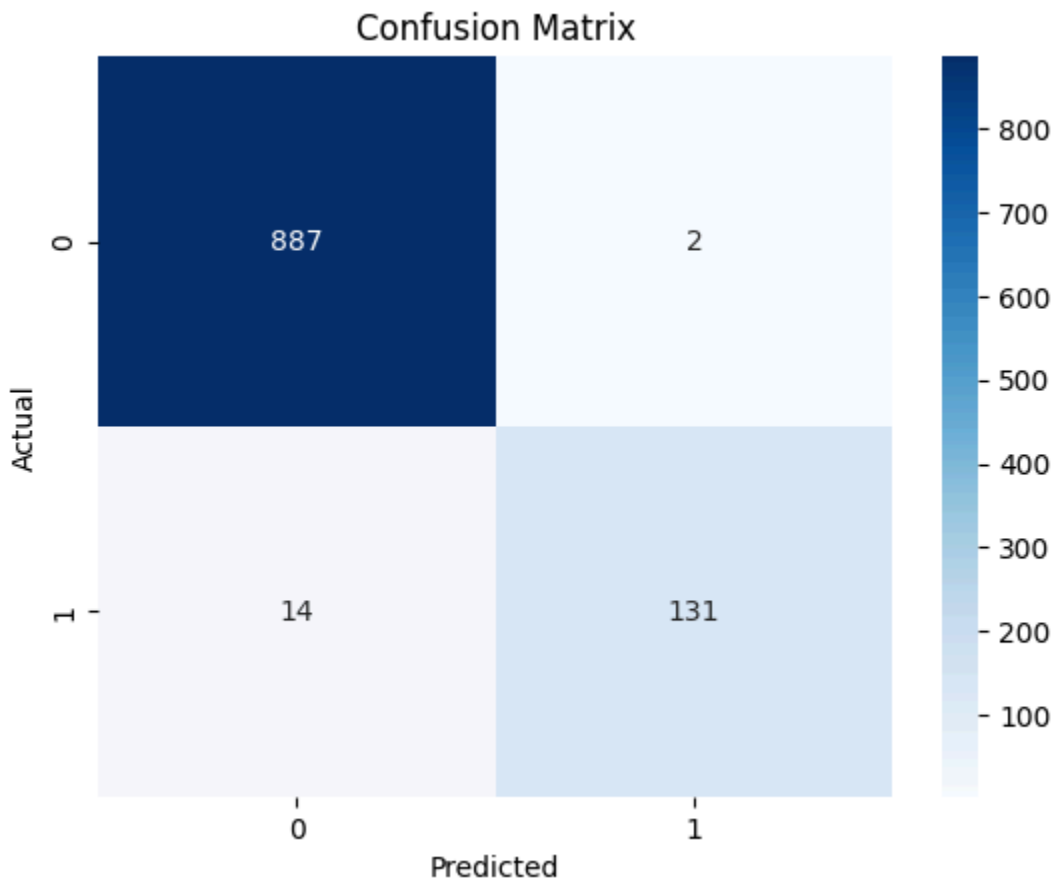
```python
model.fit(X_train, y_train)

print("\nModel Training Completed")


# ==============================
# Step 9: Predictions
# ==============================

y_pred = model.predict(X_test)


# ==============================
# Step 10: Evaluation
# ==============================

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy Score:", accuracy)

print("\nConfusion Matrix:")
print(cm)


# ==============================
# Step 11: Confusion Matrix Plot
# ==============================

plt.figure()
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
First 5 rows:
  label                                                message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5169 entries, 0 to 5168
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   label    5169 non-null   object
 1   message  5169 non-null   object
dtypes: object(2)
memory usage: 80.9+ KB
None

Dataset Shape After Removing Duplicates: (5169, 2)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```



```
Model Training Completed

Accuracy Score: 0.9845261121856866

Confusion Matrix:
[[887   2]
 [ 14 131]]
```



In [46]:

```python
# ==============================
# Step 1: Import Libraries
# ==============================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# NLP
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Feature extraction
from sklearn.feature_extraction.text import TfidfVectorizer

# Model
```

```python
from sklearn.naive_bayes import MultinomialNB

# Train/Test split
from sklearn.model_selection import train_test_split, GridSearchCV

# Evaluation
from sklearn.metrics import accuracy_score, confusion_matrix

# Download stopwords
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[46]: True

In [47]:

```python
# ==============================
# Step 2: Import Dataset
# ==============================

data = pd.read_csv("IMDB.csv")

print("First 5 rows:")
print(data.head())

print("\nDataset Info:")
print(data.info())

print("\nSentiment Distribution:")
print(data['sentiment'].value_counts())
```

```
First 5 rows:
                                                 review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   review     50000 non-null  object
 1   sentiment  50000 non-null  object
dtypes: object(2)
memory usage: 781.4+ KB
None

Sentiment Distribution:
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

In [48]:

```python
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix

# a) Import dataset
df = pd.read_csv("IMDB.csv")

print(df.head())

# b) Data preprocessing
df = df.drop_duplicates()

X = df["review"]
y = df["sentiment"]

# Convert text labels to numbers
y = y.map({"positive":1, "negative":0})

# Convert text to numerical features
vectorizer = CountVectorizer(stop_words="english")

X = vectorizer.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# c) Train Naïve Bayes model
model = MultinomialNB()

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# d) Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Hyperparameter tuning
param_grid = {"alpha":[0.1,0.5,1.0]}

grid = GridSearchCV(MultinomialNB(), param_grid, cv=5)

grid.fit(X_train, y_train)

print("Best Parameter:", grid.best_params_)
```

```
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
Accuracy: 0.8568115357466976
Confusion Matrix:
[[4319  620]
 [ 800 4178]]
Best Parameter: {'alpha': 1.0}
```

In [48]: